PREDICTING NEXT-DAY WILDFIRE SPREAD

Phase 1 Report · Machine Learning Project

Team TensorTitans · Section C · Vanshika, Riya, Krish, Nitanshu, Tejas

October 2026

Summary

We tried to predict **which 1 km × 1 km squares of land will be on fire tomorrow**, using satellite and weather data from today.

What we found, in five points:

1. **Fire is very rare.** Only 0.08% to 1.09% of squares are on fire the next day. This makes the problem hard and makes "accuracy" a useless score.
2. **"Is it burning today?" is the strongest clue.** Squares that burn tomorrow are **120 times** more likely to be burning today. Weather alone barely tells fire and no-fire apart.
3. **Our best feature works.** We counted how many neighbouring squares are on fire. Squares that burn tomorrow have **90 times** more burning neighbours on average.
4. **Our trained model did not learn it yet.** On the test set, our best model (LightGBM) caught **0 of 157** fires. A simple rule using our neighbour count did **17 times** better than random guessing.
5. **The main reason is too little data.** The model learned from only **62** fire squares. The next step is to use many more maps.

---

1. Why we chose this problem

- **It matters in real life.** Fire crews and evacuations are planned a day ahead. Knowing where a fire is today is not enough. People need to know where it will be tomorrow.
- **The data is real and public.** We used the "Next Day Wildfire Spread" dataset by Huot et al. (2022). It covers US wildfires from 2012 to 2020.
- **It is a real machine learning challenge.** Fire is rare, so we must use the right scores. Neighbouring squares look alike, so we must split the data carefully to avoid cheating.

Earlier work used machine learning to map wildfire risk, mostly from land and terrain (Tonini et al., 2020). A review of about 300 wildfire machine learning papers warns that many studies suffer from exactly these two problems: rare events and leakage between nearby data (Jain et al., 2020).

2. The data

What one sample looks like

Each sample is a **map of 64 km × 64 km**. The map is a grid of 64 × 64 = **4,096 squares**, and each square is 1 km × 1 km.

For every map we get **13 layers** stacked on top of each other: weather, land, fuel, and fire. We turned every square into **one row of a table**, so a model can read it.

The 13 columns

COLUMN	FULL NAME	UNIT	WHAT IT TELLS THE MODEL	GROUP
`elevation`	Elevation	m	How high the land is. Fire runs faster uphill.	C
`th`	Wind direction	degrees	Which way the fire is pushed	A
`vs`	Wind speed	m/s	How hard the fire is pushed	A
`tmmn`	Minimum temperature	K	How warm the night stays	A
`tmmx`	Maximum temperature	K	Daytime heat that dries fuel	A
`sph`	Specific humidity	kg/kg	Moisture in the air	A
`pr`	Precipitation	mm/day	Rain makes fuel wet	A
`pdsi`	Palmer Drought Severity Index	index	Long-term dryness (negative = drought)	C
`NDVI`	Normalized Difference Vegetation Index	index	How much green plant fuel there is	C
`population`	Population density	people/km²	Human activity and fire starts	C
`erc`	Energy Release Component	index	How much heat dry fuel would give off	C
`PrevFireMask`	Fire mask, today	0 or 1	Is this square burning today?	B
`FireMask`	**Fire mask, tomorrow**	**0 or 1**	**The answer we want to predict**	—

Temperatures are in **Kelvin**. To get °C, subtract 273 (so 296 K ≈ 23 °C).

Files and splits

The dataset comes as **19 files**. We kept every file in one group only:

GROUP	FILES	SIZE	USED FOR
Train	15	3,047 MB	Teaching the models
Validation	2	382 MB	Choosing settings

GROUP	FILES	SIZE	USED FOR
Test	2	344 MB	Final check, used once

How we got the number of rows

STEP	TRAIN	VALIDATION	TEST
1. Maps read from the files	30	10	10
2. Squares (maps × 4,096)	122,880	40,960	40,960
3. Remove squares the satellite could not see (value -1)	-5,729	-11,168	-8,093
4. Maps **with** fire: keep every fire square, plus 10 no-fire squares per fire square	682	429	1,747
5. Maps **without** fire: keep every square	76,562	25,696	12,682
Rows used (step 4 + step 5)	**77,244**	**26,125**	**14,429**
...of which on fire tomorrow	62	39	157
Share on fire	0.08%	0.15%	1.09%

Two things to notice:

- Most rows come from maps **without** fire, because the 10-to-1 sampling only applies to maps that have fire.
- **All 39 validation fires come from a single map.** One validation map could not be seen at all, so only 9 validation maps are left.

---

3. How we avoided cheating (data leakage)

"Leakage" means the model sees information it would not have in real life, which makes its score look better than it really is. We took four steps against it:

1. **Split by whole file, not by square.** Neighbouring squares look almost the same. If one square were in train and its neighbour in test, the model could just copy the answer. Keeping whole files together stops this.
2. **Only use today's data.** Every input is from day *t*. Tomorrow's data is only used as the answer.
3. **Fit the scaler on train only.** The scaler learns averages from the data. If it saw test data, test information would leak into training.
4. **Use the test set once.** We chose all settings on the validation set, then checked the test set a single time at the end.

4. What we learned from exploring the data (EDA)

Fire vs no-fire squares (train set averages)

MEASUREMENT	NO FIRE TOMORROW	FIRE TOMORROW
Max temperature (K)	296.7	299.3
Wind speed (m/s)	3.64	3.67
Humidity (kg/kg)	0.007	0.008
Fuel dryness, ERC	38.0	36.0
Burning today	**0.13%**	**16.1%**

The weather numbers are almost the same for both groups. The big difference is in the last row: squares that burn tomorrow were **120 times** more likely to be burning today. These averages come from only 62 fire squares, so small differences (like ERC) should not be over-read.

How the columns relate to each other

"r" (correlation) is a number from -1 to +1 that shows how much two columns move together. 0 means no link.

PAIR	R	WHAT IT MEANS
Min temperature & humidity	0.89	Warm air holds more water
Max & min temperature	0.87	Hot days have warm nights
Drought index & NDVI	0.61	Wetter land is greener
ERC & drought index	-0.53	Drier fuel goes with drought

These match real-world physics, which tells us the data is trustworthy. There were **0 missing values**.

Which columns matter for predicting fire

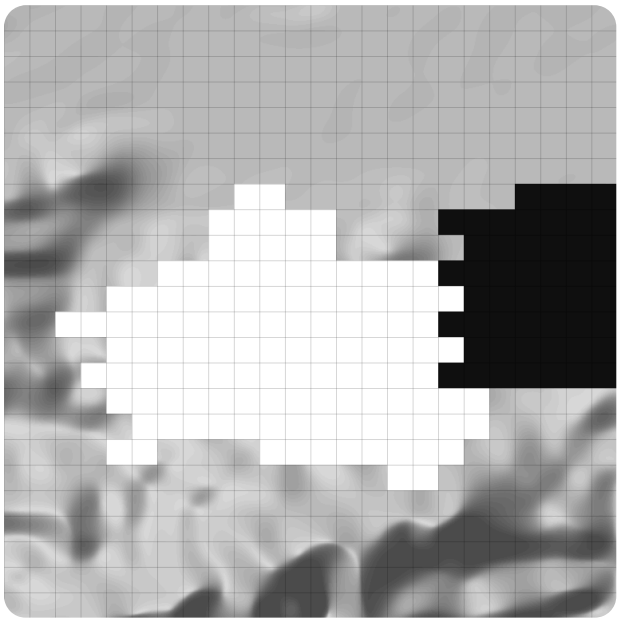
We checked each column's r with tomorrow's fire, on train and on test:

- `PrevFireMask` **(fire today): 0.12 on train and 0.31 on test.** This is the only raw column that is clearly linked to tomorrow's fire in both.
- **All 11 weather and land columns: |r| ≤ 0.015 on train.** On their own they tell us almost nothing.
- Some columns look linked on test only (elevation 0.28, ERC 0.20). This happens because the test fires sit in a few of the 10 test maps, so these columns tell us *which fire* it is, not *which square* will burn.

5. Feature engineering

We built four new columns from the raw ones:

NEW FEATURE	HOW WE MADE IT	WHY
`temp_range`	max temp - min temp	A big gap between day and night means dry air
`wind_u` , `wind_v`	wind speed × sin / cos of direction	359° and 1° both mean "north", but as numbers they look far apart. Splitting wind into east-west and north-south parts fixes this.
`drought_fuel_index`	ERC × (drought + 1)	Dry fuel **and** drought together is worse than either alone
`fire_neighbor_3x3_sum` , `fire_neighbor_5x5_sum`	Count of burning squares around each square today (8 nearest, and 24 in a wider ring)	Fire spreads to its neighbours



Real test map #5, zoomed in (each small box is 1 km). Black = burning today (50 squares). White = new fire tomorrow. The new fire appears right next to the old fire, which is why we built the neighbour count.

The neighbour count is our strongest feature:

	AVERAGE BURNING NEIGHBOURS
Squares with no fire tomorrow	0.01
Squares with fire tomorrow	0.935

That is about **90 times** higher. Its r with tomorrow's fire is 0.15 on train and 0.37 on test, higher than any raw column.

Feature groups

To see what each type of information adds, we made four growing groups:

GROUP	WHAT IT CONTAINS	NUMBER OF FEATURES
A	Weather on day *t*	6
B	A + fire today	7
C	B + land, fuel and population	12
D	C + the engineered features above	18

---

6. The models

We trained four models, from the simplest to the most powerful. Each one answers one question.

MODEL	WHAT IT DOES	THE QUESTION IT ANSWERS
Persistence	Says "tomorrow looks like today". No learning.	Does learning beat a simple rule?
Logistic Regression	Gives each column a weight and adds them up (a straight line)	Is a simple linear model enough?
LightGBM	Builds many small decision trees, each fixing the last one's mistakes	Do combinations of clues help (for example hot **and** a neighbour burning)?
XGBoost	Same idea as LightGBM, different library	Is LightGBM's result just one library's quirk?

All models used **Group D (18 features)**. Because fire is rare, we told the models that **missing a fire matters more** than a false alarm: `class_weight = balanced` for Logistic Regression, and `scale_pos_weight = 8` for LightGBM and XGBoost.

How we score the models

Why not accuracy? About 99% of squares are "no fire". A model that **always** says "no fire" gets **98.91%** accuracy on the test set, while catching zero fires. So accuracy hides failure.

We use these scores instead:

- **PR-AUC.** How well the model ranks real fires above non-fires, focusing on the rare fire class. For rare events it is more honest than ROC-AUC (Saito & Rehmsmeier, 2015). A random guess scores about the fire rate itself (for example 0.011 on test), so the score only means something when it is clearly above that.
- **ROC-AUC.** 0.5 means a coin flip and 1.0 is perfect.
- **Recall at 5% false alarms.** If we allow 5% of no-fire squares to be wrongly flagged, what share of real fires do we catch?
- **Threshold.** The model gives a score from 0 to 1. We picked the cut-off for "fire" on the validation set only, then kept it fixed for the test.

7. Results

Validation set (39 fire squares)

MODEL	PR-AUC	ROC-AUC	RESULT
Persistence	0.0015	0.498	Same as a coin flip
Logistic Regression	0.0008	0.141	Ranked fires backwards
LightGBM	**0.0016**	**0.518**	**Best → used on test**
XGBoost	0.0009	0.199	Ranked fires backwards
Random guess	*0.0015*	*0.500*	

LightGBM was the best model, and the only one above 0.5 ROC-AUC. Trees can combine clues, which a straight line cannot. But it only just beat random guessing (0.0016 vs 0.0015), so we chose it for the test, not because it was clearly good.

Why did two models score below a coin flip? In the validation set, **none of the 39 fires were burning the day before**, and all 88 squares that were burning went out. The models learned "burning today means burning tomorrow" from training, and in validation that was exactly backwards. All 39 validation fires come from one map, so one unusual fire decided these scores.

Feature groups on validation (LightGBM)

GROUP	PR-AUC	ROC-AUC
A (weather)	0.0016	0.30
B (+ fire today)	0.0019	0.55
C (+ land)	0.0022	0.66
D (+ engineered)	0.0016	0.50

PR-AUC stays near the random level for every group. ROC-AUC jumps around, but with only 39 fires from one map, these jumps are mostly noise. This validation set is too small to rank the groups.

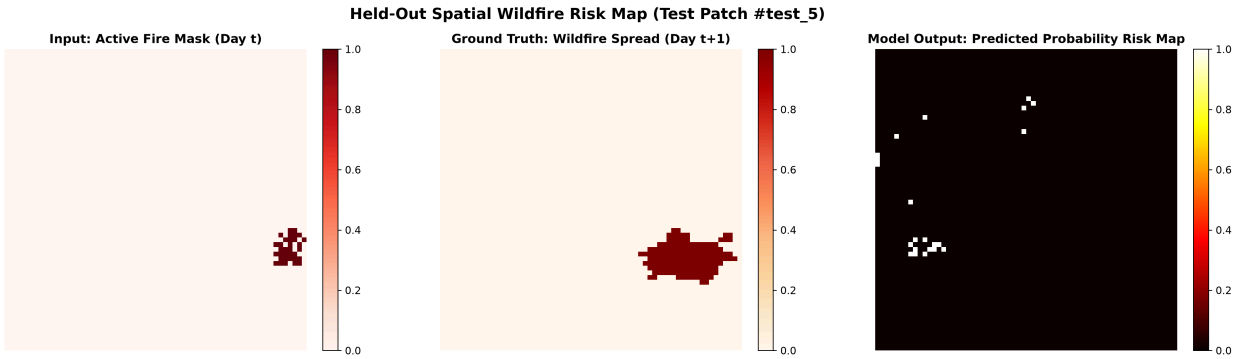
Test set (final check, used once)

SCORE	LIGHTGBM	RANDOM GUESS
PR-AUC	0.0109	0.0109
ROC-AUC	0.495	0.500
Recall at 5% false alarms	0.00	0.05
Accuracy	98.25%	98.91% (always "no fire")

Confusion matrix (what the model got right and wrong):

	REAL FIRE	REAL NO FIRE
Model said fire	0 (caught)	96 (false alarms)
Model said no fire	157 (missed)	14,176 (correct)

The model **missed all 157 real fires** and raised 96 false alarms. Its accuracy (98.25%) is even **lower** than always saying "no fire" (98.91%).



Test map #5. Left: fire today. Middle: real fire tomorrow. Right: the model's guess. It flags 22 scattered squares, and none of them is where the fire spread.

---

8. Key insights

1. The signal is in the data, but our model did not learn it

We tested two **simple rules with no training** on the same test set:

METHOD (TEST SET)	PR-AUC	ROC-AUC	VS RANDOM
Random guess	0.011	0.500	1×
Our LightGBM	0.011	0.495	1×
Rule: "burning today → burning tomorrow"	0.109	0.588	10×
Rule: "count burning neighbours"	**0.182**	**0.632**	**17×**

A one-line rule using **our** neighbour feature beats the trained model by a wide margin. So the features are useful. The model is what failed.

2. Too few fires to learn from

The model learned from only **62 fire squares** among 77,244 rows. With so few examples, the final model **collapsed**: across all 14,429 test squares it gave only **58 different output values**, so it cannot rank squares from most to least risky.

3. The cut-off was moved between two different models

The fire cut-off (0.0007) was chosen using the LightGBM from notebook 05, but it was used on a **different** LightGBM retrained in notebook 06 with other settings. Scores from two different models cannot be compared, so the cut-off does not really carry over.

4. Fire today matters most; weather alone does not

All 11 weather and land columns have almost no link to tomorrow's fire on their own ($|r| \leq 0.015$ on train). Fire today, and especially burning neighbours, carry the real signal.

5. Accuracy hides failure

98.25% accuracy sounds great, but the model caught zero fires. For rare events, always check PR-AUC, recall, and the confusion matrix.

---

9. Problems we found and fixed in our own pipeline

While checking our work, we found five problems and fixed them before producing the final numbers:

1. **Neighbour counts were wrong.** They were counted after most no-fire rows had already been removed by sampling, so many real burning neighbours were missing and the counts came out near zero. We now count on the full 64 × 64 map first. The feature's r with tomorrow's fire on train went from **0.065 to 0.154**.
2. **Map IDs repeated across splits.** Every split numbered its maps from 0, so the "no overlap between splits" check was comparing the same numbers. Map IDs now include the split name (for example `train_0` and `test_0`).
3. **The leakage check always printed "PASS".** Even when the check failed, the notebook printed `[PASS]`. It now prints PASS or FAIL based on the real result.
4. **Some written numbers did not match the outputs.** For example, a notebook claimed one PR-AUC was 0.12 when the real value was 0.0016. All write-ups now use the real computed numbers.
5. **The test result changed between runs.** LightGBM used several CPU threads, and tiny rounding differences changed which squares crossed the very small cut-off (0.00069): three identical runs flagged 499, 499 and 4 squares. We switched LightGBM to deterministic mode with one thread, and now every run gives exactly the same result.

10. Limitations

- **Small sample.** We used only 30 / 10 / 10 maps, giving 62 / 39 / 157 fire squares. This is the biggest limit on our results.
- **Unusual validation set.** All validation fires come from one map, so validation scores are noisy and partly misleading.
- **Squares are treated one by one.** A table row does not see the shape of the fire front, only counts of nearby fire.
- **Weather is coarse.** Weather values change slowly across a map, so they cannot tell neighbouring squares apart.

---

11. Next steps

1. **Use many more maps:** hundreds per split instead of 30 / 10 / 10, so the model sees hundreds of fires.
2. **Stop the model from collapsing:** use early stopping and settings that limit tree size (for example `min_child_samples`).
3. **Pick the cut-off on the same model** that is tested.
4. **Always report the simple rules** (persistence and neighbour count) on the test set as baselines.
5. **Phase 2:** a U-Net (Ronneberger et al., 2015), a neural network that looks at the whole 64 × 64 map at once and can learn the shape of the fire.

---

12. Team and contributions

MEMBER	WORK
Vanshika	Literature review, dataset audit, data loader
Riya	Exploratory data analysis
Krish	Feature engineering, error analysis
Nitanshu	Train / validation / test splits, evaluation and thresholds, final comparison
Tejas	Models, evaluation

Code: github.com/connectwithvanshika/TensorTitans_SectionC

Glossary

TERM	SIMPLE MEANING
Square / pixel	One 1 km × 1 km piece of land, one row in our table
Label	The answer we want to predict: fire tomorrow (1) or not (0)
Train / validation / test	Data to learn from / data to choose settings on / data for the final, one-time check
Leakage	When the model sees information it would not have in real life, making its score look too good
Correlation (r)	How much two columns move together, from -1 to +1. 0 means no link.
Feature	A column the model uses as input
Threshold (cut-off)	The score above which we call a square "fire"
Precision	Of the squares we called fire, how many really burned
Recall	Of the squares that really burned, how many we caught
PR-AUC	One number summarising precision and recall over all cut-offs. Random = the fire rate.
ROC-AUC	How well the model ranks fire above no-fire. 0.5 = coin flip, 1.0 = perfect.
Confusion matrix	A 2 × 2 table of caught fires, false alarms, missed fires, and correct "no fire"

References

1. F. Huot, R. L. Hu, N. Goyal, T. Sankar, M. Ihme, Y.-F. Chen. "Next Day Wildfire Spread: A Machine Learning Dataset to Predict Wildfire Spreading From Remote-Sensing Data." *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–13, 2022. doi:10.1109/TGRS.2022.3192974
2. T. Saito, M. Rehmsmeier. "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Imbalanced Datasets." *PLoS ONE*, 10(3): e0118432, 2015. doi:10.1371/journal.pone.0118432
3. P. Jain, S. C. P. Coogan, S. G. Subramanian, M. Crowley, S. Taylor, M. D. Flannigan. "A review of machine learning applications in wildfire science and management." *Environmental Reviews*, 28(4), pp. 478–505, 2020. doi:10.1139/er-2020-0019
4. M. Tonini, M. D'Andrea, G. Biondi, S. Degli Esposti, A. Trucchia, P. Fiorucci. "A Machine Learning-Based Approach for Wildfire Susceptibility Mapping. The Case Study of the Liguria Region in Italy." *Geosciences*, 10(3): 105, 2020. doi:10.3390/geosciences10030105
5. O. Ronneberger, P. Fischer, T. Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation." *MICCAI 2015*, pp. 234–241, Springer. doi:10.1007/978-3-319-24574-4_28